

How to Use This SQL Guide

Candidate reports mention SQL questions in the OA, and SQL is central to the technical interview.

Each problem gives the schema with sample data, the question, a verified solution query, expected output and an explanation. Every query was executed in SQLite.

SQL Problems

Problem 1: Second Highest Salary

Difficulty: Easy | Pattern: Subquery / DISTINCT | Authenticity: Community Reported

Schema & Sample Data

```
CREATE TABLE employee (id INTEGER, name TEXT, salary INTEGER);
INSERT INTO employee VALUES (1, 'Asha', 50000), (2, 'Ravi', 80000), (3, 'Meera', 80000), (4, 'John', 60000)
```

Question

Find the second highest distinct salary; NULL if it does not exist.

Solution Query

```
SELECT MAX(salary) AS second_highest
FROM employee
WHERE salary < (SELECT MAX(salary) FROM employee);
```

Expected Output

```
second_highest
60000
```

Explanation

Inner query finds the top salary (80000); outer takes the max below it, skipping duplicate tops -> 60000.

Problem 2: Count Employees per Department

Difficulty: Easy | Pattern: GROUP BY | Authenticity: Community Reported

Schema & Sample Data

```
CREATE TABLE emp (id INTEGER, name TEXT, dept TEXT);
INSERT INTO emp VALUES (1, 'A', 'Sales'), (2, 'B', 'Sales'), (3, 'C', 'Tech'), (4, 'D', 'Tech'), (5, 'E', 'Tech')
```

Question

Count employees per department, ordered by count desc then dept asc.

Solution Query

```
SELECT dept, COUNT(*) AS emp_count
FROM emp
GROUP BY dept
ORDER BY emp_count DESC, dept ASC;
```

Expected Output

```
dept|emp_count
Tech|3
```

```
Sales|2
HR|1
```

Explanation

GROUP BY collapses per department; COUNT(*) counts members; ORDER BY ranks them.

Problem 3: Employees Above Department Average

Difficulty: Medium | Pattern: Correlated Subquery | Authenticity: Community Reported

Schema & Sample Data

```
CREATE TABLE staff (id INTEGER, name TEXT, dept TEXT, salary INTEGER);
INSERT INTO staff VALUES (1, 'A', 'Sales', 40000), (2, 'B', 'Sales', 60000), (3, 'C', 'Tech', 90000), (4, 'D', 'Tech', 70000);
```

Question

List employees earning above their own department's average, ordered by name.

Solution Query

```
SELECT name, salary
FROM staff s
WHERE salary > (SELECT AVG(salary) FROM staff WHERE dept = s.dept)
ORDER BY name;
```

Expected Output

```
name|salary
B|60000
C|90000
```

Explanation

The correlated subquery computes each employee's department average (Sales 50000, Tech 70000).

Problem 4: Find Duplicate Emails

Difficulty: Easy | Pattern: GROUP BY + HAVING | Authenticity: Community Reported

Schema & Sample Data

```
CREATE TABLE person (id INTEGER, email TEXT);
INSERT INTO person VALUES (1, 'a@x.com'), (2, 'b@x.com'), (3, 'a@x.com'), (4, 'c@x.com'), (5, 'b@x.com');
```

Question

Find emails appearing more than once, with counts, ordered by email.

Solution Query

```
SELECT email, COUNT(*) AS occurrences
FROM person
GROUP BY email
HAVING COUNT(*) > 1
ORDER BY email;
```

Expected Output

```
email|occurrences
a@x.com|3
b@x.com|2
```

Explanation

HAVING filters aggregated groups; WHERE cannot filter on an aggregate.

Problem 5: Customers With No Orders

Difficulty: Medium | Pattern: LEFT JOIN / Anti-Join | Authenticity: Community Reported

Schema & Sample Data

```
CREATE TABLE customers (id INTEGER, name TEXT);
CREATE TABLE orders (order_id INTEGER, customer_id INTEGER);
INSERT INTO customers VALUES (1, 'Alice'), (2, 'Bob'), (3, 'Carol'), (4, 'Dan');
INSERT INTO orders VALUES (101, 1), (102, 1), (103, 3);
```

Question

List customers who never placed an order, ordered by name.

Solution Query

```
SELECT c.name
FROM customers c
LEFT JOIN orders o ON c.id = o.customer_id
WHERE o.order_id IS NULL
ORDER BY c.name;
```

Expected Output

```
name
Bob
Dan
```

Explanation

LEFT JOIN keeps all customers; NULL order columns identify those without orders.

Problem 6: Running Total of Daily Sales

Difficulty: Medium | Pattern: Window (SUM OVER) | Authenticity: Community Reported

Schema & Sample Data

```
CREATE TABLE sales (day INTEGER, amount INTEGER);
INSERT INTO sales VALUES (1, 100), (2, 150), (3, 50), (4, 200);
```

Question

Compute the cumulative running total of sales by day.

Solution Query

```
SELECT day, amount,
       SUM(amount) OVER (ORDER BY day) AS running_total
FROM sales
ORDER BY day;
```

Expected Output

```
day|amount|running_total
1|100|100
2|150|250
3|50|300
4|200|500
```

Explanation

SUM(amount) OVER (ORDER BY day) accumulates from the first row to the current one.

Problem 7: Top 2 Products by Sales per Category

Difficulty: Hard | Pattern: Window (RANK/PARTITION) | Authenticity: Community Reported

Schema & Sample Data

```
CREATE TABLE product_sales (category TEXT, product TEXT, sales INTEGER);
INSERT INTO product_sales VALUES ('A', 'p1', 100), ('A', 'p2', 200), ('A', 'p3', 150), ('B', 'p4', 300), ('B', 'p6', 400);
```

Question

For each category, return the top 2 products by sales; order by category then sales desc.

Solution Query

```
SELECT category, product, sales
FROM (
    SELECT category, product, sales,
           RANK() OVER (PARTITION BY category ORDER BY sales DESC) AS rnk
    FROM product_sales
) ranked
WHERE rnk <= 2
ORDER BY category, sales DESC;
```

Expected Output

```
category|product|sales
A|p2|200
A|p3|150
B|p6|400
B|p4|300
```

Explanation

RANK() with PARTITION BY restarts ranking per category; keep ranks 1-2.

Problem 8: Total Revenue per Customer (JOIN + GROUP BY)

Difficulty: Medium | Pattern: JOIN + Aggregation | Authenticity: Research-Backed Company Style

Schema & Sample Data

```
CREATE TABLE cust (id INTEGER, name TEXT);
CREATE TABLE ord (oid INTEGER, cid INTEGER, amt INTEGER);
INSERT INTO cust VALUES (1, 'Ann'), (2, 'Ben'), (3, 'Cara');
INSERT INTO ord VALUES (1, 1, 200), (2, 1, 300), (3, 2, 500), (4, 3, 100), (5, 3, 400);
```

Question

Show each customer's total revenue, highest first, then name.

Solution Query

```
SELECT c.name, SUM(o.amt) AS revenue
FROM cust c
JOIN ord o ON c.id = o.cid
GROUP BY c.name
ORDER BY revenue DESC, c.name;
```

Expected Output

```
name|revenue
Ann|500
Ben|500
Cara|500
```

Explanation

Join then aggregate: Ann 200+300, Ben 500, Cara 100+400 all equal 500; ties broken alphabetically by name (Ann, Ben, Cara).

Tredence

SQL Problems

Applied Role(s):

[Analyst - Data Scientist](#)

Online Assessment Preparation Material

Version 1.1 • Last Updated: July 2026

Every question carries an authenticity label. See the legend on the next page.

Table of Contents

How to Use This SQL Guide	1
SQL Problems	1
Problem 1: Second Highest Salary	1
Problem 2: Count Employees per Department	1
Problem 3: Employees Above Department Average	2
Problem 4: Find Duplicate Emails	2
Problem 5: Customers With No Orders	3
Problem 6: Running Total of Daily Sales	3
Problem 7: Top 2 Products by Sales per Category	3
Problem 8: Total Revenue per Customer (JOIN + GROUP BY)	4
Authenticity Legend & How to Read This Document	7

Authenticity Legend & How to Read This Document

Every question and problem in this document is labelled with an **authenticity classification** so you know how closely it maps to what candidates have actually reported. Labels are assigned conservatively based on research; authored practice questions are never presented as real exam questions.

Label	Meaning
Verified Previous-Year	The specific problem/pattern is documented from actual reported assessment experiences.
Frequently Repeated	The topic or pattern type is documented as recurring across multiple hiring cycles.
Community Reported	A widely shared, role-relevant pattern reported across placement communities and forums.
Research-Backed Company Style	Authored to match the researched difficulty, style and pattern - practice, not a real past question.

The correct option in every MCQ is highlighted in green and also restated below the question. Study the pattern, not just the answer.